

A compact numerical implementation for solving Stokes equations using matrix-vector operations

Tao Zhang^{1,2}, Amgad Salama¹, Shuyu Sun¹, Hua Zhong^{1*}

¹*Computational Transport Phenomena Laboratory (CTPL), King Abdullah University of Science and Technology (KAUST), Thuwal 23955-6900, KSA*

²*National Engineering Laboratory for Pipeline Safety, Beijing Key Laboratory of Urban Oil and Gas Distribution Technology, China University of Petroleum, Beijing, 102249, China*

**Email: Hua.Zhong@kaust.edu.sa*

Abstract

In this work, a numerical scheme is implemented to solve Stokes equations based on cell-centered finite difference over staggered grid. In this scheme, all the difference operations have been vectorized thereby eliminating loops. This is particularly important when using programming languages that require interpretations, e.g., MATLAB and Python. Using this scheme, the execution time becomes significantly smaller compared with non-vectorized operations and also become comparable with those languages that require no repeated interpretations like FORTRAN, C, etc. This technique has also been applied to Navier-Stokes equations under laminar flow conditions.

Keywords: Stokes flow; Cell-centered finite difference; Vectorized differential operators; MATLAB

1 Introduction

Every aspect of life around us involves fluids that are either static or dynamic. No wonder, therefore, they attract our attention to capture the roles that control their dynamics. Since the establishment of the well-known Navier-Stokes equations around the mid of the eighteenth century, significant amount of research work have been conducted and still to solve these equations under different assumptions and boundary conditions. One of the simplifications that have been recognized are those related to a class of flow that is very slow (creeping flow) where the Reynolds number is much smaller than one. Under such conditions George Stokes (1819-1903) assumed that the advective term, which accounts for fluid inertia, may be neglected and the equations reduce to what is called Stokes flow. This assumption significantly simplifies the solution by dropping the troublesome nonlinear advection. Stokes flow can occur in many engineering applications, including flow in porous media. As such a great deal of interest has been devoted to the development of solution strategies that

are fast and accurate. Although there has been success in finding analytical solution in simple geometries and boundary conditions, in general cases the solution will be thought of numerically.

Recently, there have been a number of computer languages gained popularity for their ease of handling and coding, e.g., MATLAB, Python and the like. The drawback of these languages, however, is that they require interpretations during code execution, which consumes valuable CPU time. In order to utilize these languages to their full potential, vectorizing the different operations may be essential when possible. Matrix operations have been implemented in these languages at lower level and are therefore fast. In this work, we introduce a full vectorizing algorithm to solving Stokes equations. We show that the execution time using this algorithm compares very well with codes written in other non-interpreting languages (e.g., FORTRAN, C). Using the compact notation of these languages (e.g., MATLAB), the number of lines of codes can reduce significantly. We show that the results obtained by this algorithm are as accurate and efficient as those computed using different methods, including MAC, Simple, LBM (D2Q9) in Fortran language. We also compare the execution time with that required when using traditional techniques in MATLAB in which the time consuming loops are used. Sun et al. [1] and Salama et al. [2, 3] have introduced the concept of shifting matrices to vectorize the different operations pertinent to the partial differential equations governing flow of single and two-phase system in porous media. In this work, we generalize this concept to Stokes flow in pure fluid systems with periodic boundary conditions. We also introduce exclusion matrices to account for in-domain solid regions and to generalize to other solid boundaries.

2 Numerical algorithm

To begin with, Stokes flow, also named creeping flow or creeping motion, Kim and Karrila [4] is a type of fluid flow where advective inertial forces are small compared with viscous forces, Kirby [5]. In another word, the Reynolds number, Re , is quite small, i.e. $Re \ll 1$. The equations of motion for Stokes flow, called the Stokes Equations, are a simplification of Navier-Stokes equations. According to the concept of Stokes flow, the inertial forces are assumed to be negligible compared with the viscous forces. Thus, in the common case of incompressible Newtonian fluids, the momentum balance in the Stokes Equations can be written as:

$$\mu \nabla^2 \mathbf{u} - \nabla p + \mathbf{f} = 0, \quad (1)$$

where $\mathbf{u}$ is the velocity of the fluid, ∇p is the gradient of the pressure, μ is the dynamic viscosity, and $\mathbf{f}$ is an applied body force. The governing conservation equations also include an equation for the conservation of mass, commonly written in the form:

$$\nabla \cdot \mathbf{u} = 0. \quad (2)$$

There are a number of algorithms available to solve the above system of equations numerically. In this work we will solve the two equations in one large system involving the unknown pressure and velocity fields. In matrix form (for 2D problems) this can be written as

$$\begin{pmatrix} A_{xx} & 0 & A_x \\ 0 & A_{yy} & A_y \\ A_x^T & A_y^T & 0 \end{pmatrix} \begin{pmatrix} \mathbf{u}_x \\ \mathbf{u}_y \\ p \end{pmatrix} = \begin{pmatrix} \mathbf{b}_x \\ \mathbf{b}_y \\ \mathbf{0} \end{pmatrix}. \quad (3)$$

The first row of the above matrix comes from the discretization of the x-momentum equation, the second row is for the y-momentum equation and the third row accounts for the continuity equation.

2.1 Discretization

For a steady-state Stokes flow with periodic boundary conditions, the momentum and continuity equations in Cartesian coordinates can be written in component form as:

$$\mu \frac{\partial^2 u_i}{\partial x_j \partial x_j} = \frac{\partial p}{\partial x_i} - \rho g_i, \quad (4)$$

$$\frac{\partial u_i}{\partial x_i} = 0, \quad (5)$$

in $\Omega = (0, L_x) \times (0, L_y)$ and $i=1, 2, 3$ in 3D. The periodic boundary conditions can be formulated as:

$$u_i(0, y) = u_i(L_x, y), \quad 0 < y < L_y, \quad (6)$$

$$u_i(x, 0) = u_i(x, L_y), \quad 0 < x < L_x, \quad (7)$$

$$p(0, y) = p(L_x, y), \quad 0 < y < L_y, \quad (8)$$

$$p(x, 0) = p(x, L_y), \quad 0 < x < L_x. \quad (9)$$

For the sake of simplicity without loss of generality, we use uniform mesh as that shown in Fig. 1 below. In staggered grid, the pressure is defined in the centre of the cells and the velocity at the face centre. The size of each grid could be defined as $h_x = \frac{x_m - x_0}{m}$; $h_y = \frac{y_n - y_0}{n}$.

For the unknown pressure, we define

$$\begin{pmatrix} p_{\frac{1}{2}, \frac{1}{2}}^h & p_{\frac{1}{2}, \frac{3}{2}}^h & \cdots & p_{\frac{1}{2}, n-\frac{1}{2}}^h \\ p_{\frac{3}{2}, \frac{1}{2}}^h & p_{\frac{3}{2}, \frac{3}{2}}^h & \cdots & p_{\frac{3}{2}, n-\frac{1}{2}}^h \\ \vdots & \vdots & \ddots & \vdots \\ p_{m-\frac{1}{2}, \frac{1}{2}}^h & p_{m-\frac{1}{2}, \frac{3}{2}}^h & \cdots & p_{m-\frac{1}{2}, n-\frac{1}{2}}^h \end{pmatrix} = \left(p_{i-\frac{1}{2}, j-\frac{1}{2}}^h \right) \stackrel{\text{def}}{=} (p^h) \quad (10)$$

with $i = 1, \dots, m$, $j = 1, \dots, n$. Likewise the unknown velocity can be written as

$$\text{In x-direction:} \quad \begin{pmatrix} u_{\frac{1}{2}, \frac{1}{2}}^{x,h} & u_{\frac{1}{2}, \frac{3}{2}}^{x,h} & \cdots & u_{\frac{1}{2}, n-\frac{1}{2}}^{x,h} \\ u_{\frac{3}{2}, \frac{1}{2}}^{x,h} & u_{\frac{3}{2}, \frac{3}{2}}^{x,h} & \cdots & u_{\frac{3}{2}, n-\frac{1}{2}}^{x,h} \\ \vdots & \vdots & \ddots & \vdots \\ u_{m-\frac{1}{2}, \frac{1}{2}}^{x,h} & u_{m-\frac{1}{2}, \frac{3}{2}}^{x,h} & \cdots & u_{m-\frac{1}{2}, n-\frac{1}{2}}^{x,h} \end{pmatrix} = \left(u_{i-\frac{1}{2}, j-\frac{1}{2}}^{x,h} \right) \stackrel{\text{def}}{=} [u^{x,h}]. \quad (11)$$

Similarly, the velocity in y-direction could be written as

$$\left(u_{i-\frac{1}{2}, j}^{y,h} \right) \stackrel{\text{def}}{=} (u^{y,h}). \quad (12)$$

Now discretizing the momentum equation, for that consider the momentum equation in the x-direction

$$\mu \left(\frac{\partial^2 u_x}{\partial x^2} + \frac{\partial^2 u_x}{\partial y^2} \right) = \frac{\partial p}{\partial x} - \rho g_x. \quad (13)$$

In staggered grid system, this equation is discretized at edges, such that

$$\mu \left[\frac{\partial}{\partial x} \left(\frac{\partial}{\partial x} \right) + \frac{\partial}{\partial y} \left(\frac{\partial}{\partial y} \right) \right] [u^{x,h}] = \frac{\partial}{\partial x} \left[p_{i,j-\frac{1}{2}}^h \right] - \rho g_x. \quad (14)$$

The above expression indicates that the derivative of the pressure is required at locations where it is not defined. Similarly, the momentum equation in the y-direction can be written as

$$\mu \left[\frac{\partial}{\partial x} \left(\frac{\partial}{\partial x} \right) + \frac{\partial}{\partial y} \left(\frac{\partial}{\partial y} \right) \right] [u^{y,h}] = \frac{\partial}{\partial x} \left[p_{i-\frac{1}{2},j}^h \right] - \rho g_y. \quad (15)$$

Furthermore, the continuity equation can be written as

$$\frac{\partial}{\partial x} \left[u_{i-\frac{1}{2},j-\frac{1}{2}}^{x,h} \right] + \frac{\partial}{\partial y} \left[u_{i-\frac{1}{2},j-\frac{1}{2}}^{y,h} \right] = 0. \quad (16)$$

Again, the derivatives of velocity components are evaluated at locations where the velocity is not defined.

2.2 The matrix-vector notation

To shift the information of the velocity from the centre of the cell to the edges where it is defined, we define the following shifting operators:

$$\left(S_{\frac{1}{2}}^x \right) u_{i-\frac{1}{2},j-\frac{1}{2}}^{x,h} \rightarrow u_{i,j-\frac{1}{2}}^{x,h}, \left(S_{\frac{1}{2}}^x \right) u_{i-\frac{1}{2},j-\frac{1}{2}}^{x,h} \rightarrow u_{i-1,j-\frac{1}{2}}^{x,h}, \left(S_{\frac{1}{2}}^y \right) u_{i-\frac{1}{2},j-\frac{1}{2}}^{y,h} \rightarrow u_{i-\frac{1}{2},j}^{y,h}, \left(S_{\frac{1}{2}}^y \right) u_{i-\frac{1}{2},j-\frac{1}{2}}^{y,h} \rightarrow u_{i-\frac{1}{2},j-1}^{y,h}.$$

Such shifting operators have the properties that: $S_{\frac{1}{2}} S_{\frac{1}{2}} = S_1$, $S_{-\frac{1}{2}} S_{-\frac{1}{2}} = S_{-1}$, and $S_{-\frac{1}{2}} S_{\frac{1}{2}} = S_{\frac{1}{2}} S_{-\frac{1}{2}} = I$, where I is the identity matrix. The gradient at the x-edges could, therefore, be written as

$$\frac{\partial}{\partial x_i} = \frac{S_{\frac{1}{2}}^x - S_{-\frac{1}{2}}^x}{h_{x_i}}, \quad (17)$$

with which, the momentum equation can transfer to (in x direction):

$$\mu \left[\left(\frac{S_{\frac{1}{2}}^x - S_{-\frac{1}{2}}^x}{h_x} \right) \left(\frac{S_{\frac{1}{2}}^x - S_{-\frac{1}{2}}^x}{h_x} \right) + \left(\frac{S_{\frac{1}{2}}^y - S_{-\frac{1}{2}}^y}{h_y} \right) \left(\frac{S_{\frac{1}{2}}^y - S_{-\frac{1}{2}}^y}{h_y} \right) \right] [u^{x,h}] = \left(\frac{S_{\frac{1}{2}}^x - S_{-\frac{1}{2}}^x}{h_x} \right) S_{\frac{1}{2}}^x [p^h] - \rho g_x. \quad (18)$$

To simplify the computations, we can add an operator $S_{-\frac{1}{2}}^x S_{\frac{1}{2}}^x$ to the first part of left hand side and an operator $S_{-\frac{1}{2}}^y S_{\frac{1}{2}}^y$ to the second part of left hand side. Then we can get

$$\mu \left[\left(\frac{I - S_{-\frac{1}{2}}^x}{h_x} \right) \left(\frac{S_{\frac{1}{2}}^x - I}{h_x} \right) + \left(\frac{I - S_{-\frac{1}{2}}^y}{h_y} \right) \left(\frac{S_{\frac{1}{2}}^y - I}{h_y} \right) \right] [u^{x,h}] = \left(\frac{S_{\frac{1}{2}}^x - I}{h_x} \right) [p^h] - \rho g_x. \quad (19)$$

Next, we define matrix as follows,

$$D_{cx} \stackrel{\text{def}}{=} \frac{I - S_{-\frac{1}{2}}^x}{h_x}, \quad (20)$$

$$D_{xc} \stackrel{\text{def}}{=} \frac{S_{\frac{1}{2}}^x - I}{h_x} = -D_{cx}^T. \quad (21)$$

These operators can be constructed in MATLAB in just a simple line, thus for a Cartesian grid of

size $m \times n$ the following line constructs the D_{cx} :

$$D_{cx} = (\text{speye}(m*n) - \text{kron}(\text{speye}(n), \text{circshift}(\text{eye}(m), [1,0]))) / h_x;$$

For the sake of illustration, we show what the operator, D_{cx} , look like for a 3x3 grid and what it does to the x-component velocity field with periodicity shown in Fig. 1 below.

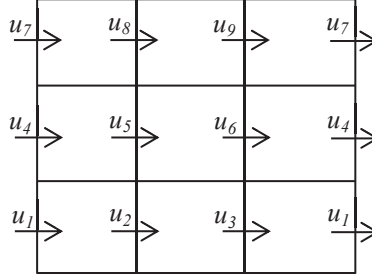


Fig. 1 X-component velocity field in a 3x3 grid

$$D_{cx} \mathbf{u} = \begin{bmatrix} 1 & 0 & -1 & 0 & 0 & 0 & 0 & 0 & 0 \\ -1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & -1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & -1 & 0 & 0 & 0 \\ 0 & 0 & 0 & -1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & -1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & -1 \\ 0 & 0 & 0 & 0 & 0 & 0 & -1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 & 1 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \\ u_5 \\ u_6 \\ u_7 \\ u_8 \\ u_9 \end{bmatrix} = \begin{bmatrix} u_1 - u_3 \\ u_2 - u_1 \\ u_3 - u_2 \\ u_4 - u_6 \\ u_5 - u_4 \\ u_6 - u_5 \\ u_7 - u_9 \\ u_8 - u_7 \\ u_9 - u_8 \end{bmatrix}. \quad (22)$$

Likewise, the operator D_{xc} when operating on the pressure field of the same 3x3 system, shown in Fig. 2 below yields the difference

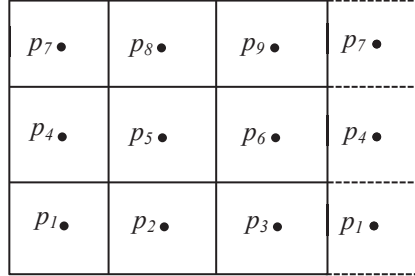


Fig. 2 Pressure field over a 3x3 grid with periodic boundary conditions

$$\begin{aligned} -D_{cx}^T \mathbf{p} &= -D_{cx}^T [p_1 \ p_2 \ p_3 \ p_4 \ p_5 \ p_6 \ p_7 \ p_8 \ p_9]^T \\ &= [p_2 - p_1 \ p_3 - p_2 \ p_1 - p_3 \ p_5 - p_4 \ p_6 - p_5 \ p_4 - p_6 \ p_8 - p_7 \ p_9 - p_8 \ p_7 - p_9]^T, \quad (23) \\ D_{sq} &= D_{cx} D_{cx}^T + D_{cy} D_{cy}^T. \quad (24) \end{aligned}$$

Therefore, the discretized momentum equation in the x-direction can be written as

$$D_{sq}[u^{x,h}] - D_{cx}^T[p^h] = \rho g_x. \quad (25)$$

Similarly, we can get the momentum equation in the y-direction

$$D_{sq}[u^{y,h}] - D_{cy}^T[p^h] = \rho g_y. \quad (26)$$

And the continuity equation is

$$D_{cx}[u^{x,h}] + D_{cy}[u^{y,h}] = 0. \quad (27)$$

And the whole system can be collected in one global matrix as:

$$\begin{bmatrix} D_{sq} & 0 & -D_{cx}^T \\ 0 & D_{sq} & -D_{cy}^T \\ D_{cx} & D_{cy} & 0 \end{bmatrix} \begin{bmatrix} u_x \\ u_y \\ p \end{bmatrix} = \begin{bmatrix} \rho g_x \\ \rho g_y \\ 0 \end{bmatrix}. \quad (28)$$

2.3 The solid boundary

As explained earlier this technique has been designed mainly for gravity-driven systems with boundary conditions that are periodic. This implies that, no specific values of boundary conditions need to be imposed. For other systems, which are more popular (e.g., pressure-driven systems), the boundaries are assigned particular velocity or pressure values. In this case, special treatments need to be devised. There are two issues here; the first is the need to assign particular cells certain predefined value of pressure or velocity; and the second is to extend the domain boundaries such that, in the extended system, the boundaries are periodic. That is to encompass the computational domain of interest within a larger region that is periodic. If the domain contains inactive cells, i.e., cells that are either assigned predefined velocity or pressure conditions or solid region inside the computational domain, one can eliminate them from the set of unknowns by using an exclusion operator. To use our scheme, the domain is extended along the right and left boundaries and therefore the boundary cells are now inside the computational domain. Such cells are excluded from the matrix of coefficients and are moved to the right hand side using exclusion operators.

As an example, for the solid boundary where the velocity is known, both the velocity and the pressure in these cells need to be removed. The solid can be written in a Boolean matrix as all the elements are zero other than the occupied cells. In matrix operation, we can have a coefficient matrix which can remove the velocity and pressure in certain cells. The restriction matrix for solid phase R should be added to remove the solid from fluid, then we can have

$$A = R \begin{bmatrix} D_{sq} & 0 & -D_{cx}^T \\ 0 & D_{sq} & -D_{cy}^T \\ D_{cx} & D_{cy} & 0 \end{bmatrix} R^T. \quad (29)$$

And therefore the global system may be written as

$$A \begin{bmatrix} u_R^{x,h} \\ u_R^{y,h} \\ p_R^h \end{bmatrix} = R \begin{bmatrix} \rho g_x \\ \rho g_y \\ 0 \end{bmatrix}. \quad (30)$$

3 Coding in MATLAB

In MATLAB, we can easily construct the discretized difference operators corresponding to the continuous operators given in equations 4 and 5 using the functions “circshift” and “kron”. Assuming a grid mesh of size $m \times n$, the difference operators defined earlier can be constructed as:

$$\begin{aligned} Dcx &= (\text{speye}(m*n) - \text{kron}(\text{speye}(n), \text{circshift}(\text{eye}(m), [1, 0]))) / hx; \\ Dcy &= (\text{speye}(m*n) - \text{kron}(\text{circshift}(\text{eye}(n), [1, 0]), \text{speye}(m))) / hy; \\ Dsq &= \mu * (Dcx * Dcx' + Dcy * Dcy'); \end{aligned}$$

To treat boundary conditions (taking channel flow as an example), we first define the boundary values using the following Boolean operators: “isPreBdry” for pressure boundary, “isUx0Bdry” for velocity boundary in x-direction and “isUy0Bdry” for velocity boundary in y-direction. They are defined (for the case of channel flow) in the following manner:

$$\begin{aligned} \text{isPreBdry} &= \text{false}(m, n); \text{isPreBdry}(1, :) = 1; \text{isPreBdry}(\text{end}, :) = 1; \text{isUx0Bdry} = \text{false}(m, n); \\ \text{isUx0Bdry}(:, 1) &= 1; \text{isUx0Bdry}(:, \text{end}) = 1; \text{isUy0Bdry} = \text{false}(m, n); \text{isUy0Bdry}(1, :) = 1; \\ \text{isUy0Bdry}(\text{end}, :) &= 1; \text{isUy0Bdry}(:, 1) = 1; \text{isUy0Bdry}(:, \text{end}) = 1; \end{aligned}$$

To construct the reduced matrix for the pressure, we use:

$$Rp = \text{speye}(m*n); Rp(\text{isPreBdry}(:, :)) = [];$$

Likewise, for the velocity, we define the reduced matrix as:

$$\begin{aligned} \text{isUx0} &= \text{isUx0Bdry} | \text{circshift}(\text{isUx0Bdry}, [-1 \ 0]); \text{isUy0} = \text{isUy0Bdry} | \text{circshift}(\text{isUy0Bdry}, [0 \ -1]); \\ Rx &= \text{speye}(m*n); Rx(\text{isUx0}(:, :)) = []; Ry = \text{speye}(m*n); Ry(\text{isUy0}(:, :)) = []; \end{aligned}$$

And, therefore, the whole R could be represented by $R = \text{blkdiag}(Rx, Ry, Rp)$;

Now, to import the boundary condition to our Matrix Equation, or to make the right hand side non-zero, we could add a matrix E to the coefficient matrix as:

$$\begin{aligned} E &= \text{sparse}(m*n, m*n); E(\text{isPreBdry}(:, :), \text{isPreBdry}(:, :)) = 1; \\ A &= [Dsq, OO, -Dcx' + E; OO, Dsq, -Dcy' + E; -Dcx, -Dcy, E]; A = R * A * R' \end{aligned}$$

where OO is the zero matrix. Thus, for the right-hand-side, we need to add

$$p = p(:); uxx = (-Dcx' + E) * p; uyy = (-Dcy' + E) * p; pp = E * p; Rhsb = [uxx; uyy; pp];$$

where p is the initial pressure we set(including the boundary) and $Rhsb$ is a vector that should be add to the right hand side:

$$bf = \text{ones}(m*n, 1); b = \rho * [gx * bf; gy * bf; 0 * bf]; b = b + Rhsb; b = R * b;$$

4 Implementation and comparisons

We apply this method to two typical steady-state benchmark problems. These include the lid driven cavity problem and the channel flow and compare the simulation results with the benchmark.

In lid driven cavity, we set the velocity of the lid to unity, the density and viscosity are 1 and there is no gravity. The simulation results show patterns as expected. The velocity of the lid drives the fluid to circulate within the computational domain as depicted in Fig. 3. We further compare the results at the centerline with those computed using traditional MAC ([6], [7], [8], [9], [10]) method and generally good agreement is achieved as shown in Fig. 4. We also check the simulation of standard channel flow as shown in Fig. 5, which shows the velocity profile and contours similar to typical channel flow.

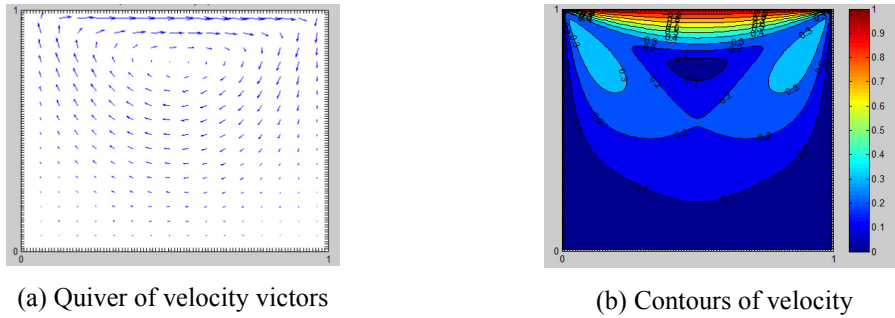


Fig. 3 Result of the shifting matrix algorithm for lid driven cavity

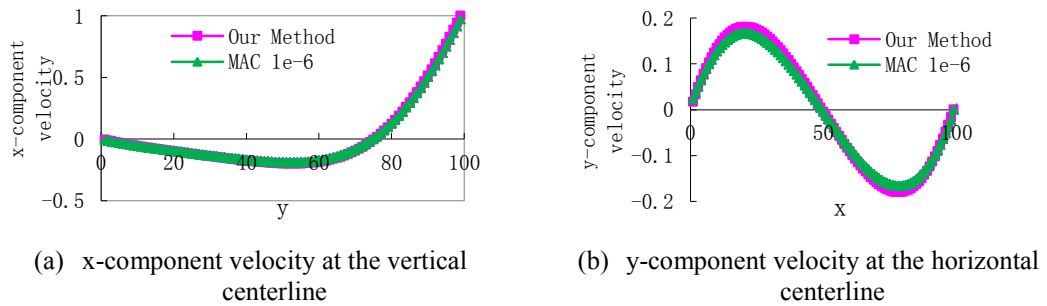


Fig. 4 Comparison of velocity components at the vertical and horizontal centerlines using our technique and MAC

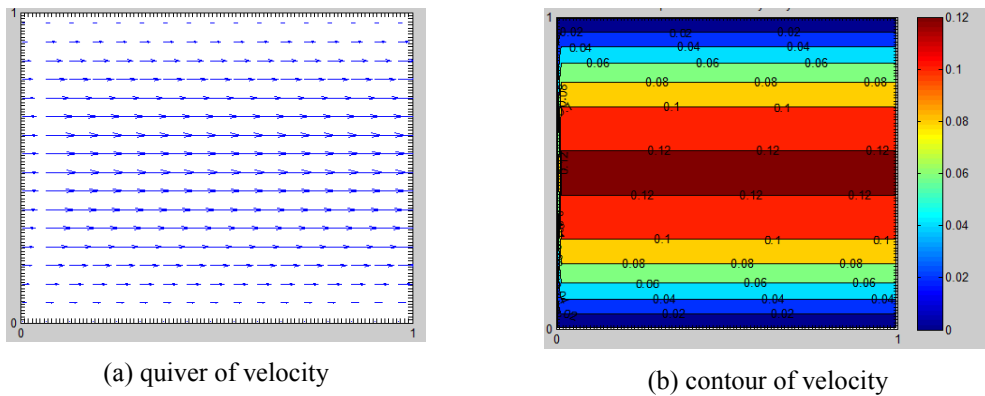


Fig. 5 Result of our method for channel flow

5 Expanding to Navier-Stokes equations

Also, this method could be extended to solve the Navier-Stokes equations. We need to add the nonlinear term and the unsteady term to the momentum equations. As an example, the discretized momentum equation in x direction using our matrix-vector notation should be like

$$\rho \frac{[u^{x,h}]^{n+1} - [u^{x,h}]^n}{\Delta t} + [u^{x,h}]^n * D_{cx}[u^{x,h}]^{n+1} + \left[u_{i,j-\frac{1}{2}}^y \right]^n * D_{cy}[u^{x,h}]^{n+1} - D_{sq}[u^{x,h}]^{n+1} - D_{cx}^T[p^h]^{n+1} = \rho g_x. \quad (31)$$

For example, for lid driven cavity, with $\Delta t = 0.01$ and number of time steps of 100, we get the result shown in Fig. 6.

6 CPU time comparisons

To show the efficiency of our method with respect to CPU time, we compare with other methods according to the following rules:

1. All the methods compared here are coded by one person, which is to avoid the effect of differences in the coding custom.
2. To acquire the CPU time more accurately, the code calls respective functions in the languages used. For example, in MATLAB, we use “tic” at the beginning of the code and “toc” at the end. In FORTRAN, we use “call cputime(a)” at the beginning and “call cputime(b)” at the end. The duration is “b-a”.

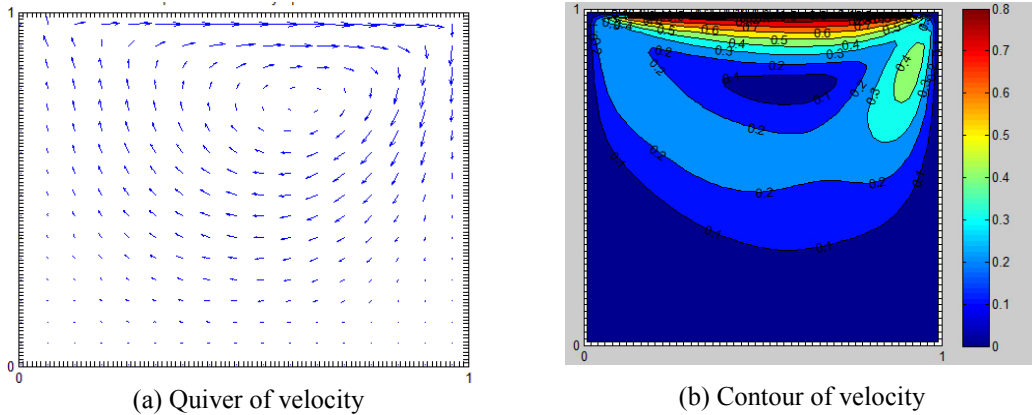


Fig. 6 Result of our method for lid driven cavity under Navier-Stokes equations

3. All the codes will present the error of each time step on the screen, and output the result to a “dat” file at the end.

We use three methods in Fortran: MAC is a finite-difference method, Simple is a finite-volume method and D2Q9 is a Lattice-Boltzmann method. Also, we draft a MATLAB code, with traditional “for” loops to calculate the coefficients of the partial difference equations one by one. For each method, we simulate the lid driven cavity problem, with mesh sizes of 10×10 to 100×100 , and get the time used to get the steady state individually. The result is as follows,

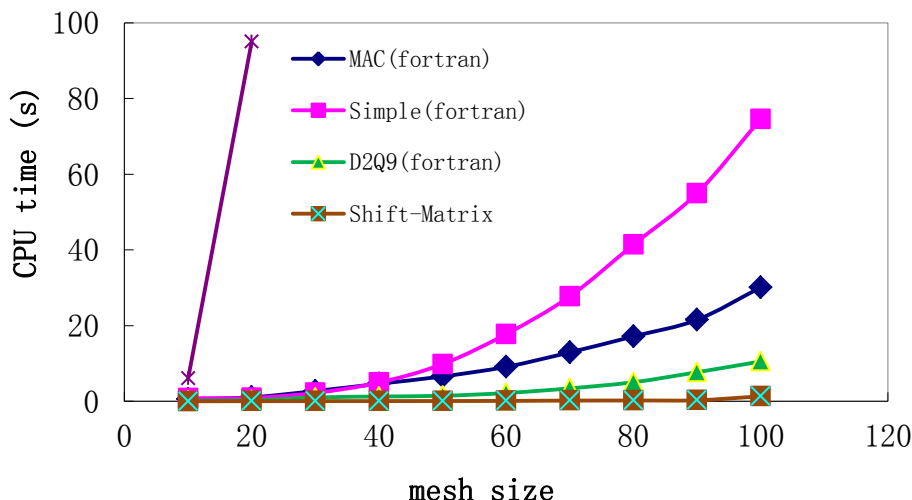


Fig. 7 Comparisons of the time used to get the steady state solution with different mesh size

Figure 7 shows that our approach (the shifting matrix algorithm), is very efficient and compares very well with many of the standard techniques utilizing non-interpreting languages.

7 Conclusion

Employing the MATLAB language's own functions, we can get a compact implementation of Stokes Equations. It's obvious that this will reduce greatly the lines of our code. With several numerical simulations of different typical cases, we proved that our method with matrix-vector notation can treat both the Stokes Equations and Navier-Stokes Equations well. Especially, for the Stokes equation, we can get the steady state solution quickly. Moreover, we can apply this method to treat more kinds of partial difference equations, such as Cahn-Hilliard equation. We can draft this method as our own function, and call it to solve other equations when needed.

References

- S. Sun, A. Salama, M.F. El Amin, *Matrix-oriented implementation for the numerical solution of the partial differential equations governing flows and transport in porous media*, Computers & Fluids, 68, (2012) 38–46
- A. Salama, S. Sun, and M.F. El-Amin, *A Novel Numerical Approach for the Solution of the Problem of Two-Phase, Immiscible Flow in Porous Media: Application to LNAPL and DNAPL*, 4th International Conference on Porous Media and its Applications in Science, Engineering and Industry, June 17-22, 2012, Potsdam, Germany, AIP Conference Proceedings, Vol. 1453, Issue 1, Pages 135-, 140
- A. Salama, A., S. Sun, M.F. El-Amin, *An Efficient Implicit-Pressure Explicit-Saturation Based Shifting-Matrix Algorithm To Simulate Two-Phase, Immiscible Flow in Porous Media With Application to CO2 Sequestration in the Subsurface*, SPE Journal, 13, 6, (2013) 1092-1101

- S. Kim, S.J. Karrila, *Microhydrodynamics: Principles and Selected Applications*, Dover. (2005) ISBN 0-486-44219-5
- B. J. Kirby, *Micro- and Nanoscale Fluid Mechanics: Transport in Microfluidic Devices*, Cambridge University Press. (2010) ISBN 978-0-521-11903-0
- J. P. Wang, A. G. L. Borthwick, R.E., Taylor, *Finite-volume-type VOF method on dynamically adaptive quadtree grids*, Int J Numer Methods Fluids, 45, (2004), 485–508
- S. McKee, M.F. Tome, *The MAC method*. Computers & Fluids 37 (2008) 907–930
- R. B., Bird, R.C. Armstrong, O. Hassager, *Dynamics of polymeric liquids. Fluid mechanics*, 2nd ed., vol. I. New York: John Wiley & Sons; 1967
- J. M. Boulton-Stone, J. R. Blake, *Gas bubbles bursting at a free surface*, J Fluid Mech, 254, (1993) 437–466
- J. U. Brackbill, H. M. Ruppel, *FLIP: a method for adaptively zoned, particle-in-cell calculations of fluid flows in two dimensions*, J Comput Phys; 65, (1986), 314–43
- J. U. Brackbill, *The ringing instability in particle-in-cell calculations of low-speed flow*, J Comput Phys, 75, (1988) 469–492
- J. U. Brackbill, *FLIP MHD: a particle-in-cell method for magnetohydrodynamics*, J Comput Phys, 96, (1991) 163–92

Appendix

%%% for Lid Driven Cavity

```
clear all; close all;
tic;
Lx=1.0; Ly=1.0; gx=0; gy=0; rho=1; mu=1; m=100;n=100; hx=Lx/m;hy=Ly/n;
isSolid=false(m,n);isSolid(m:m,1:n)=1;isSolid(1:m,1:1)=1;isSolid(1:1,1:n)=1;isSolid(1:m,n:n)=1;
Dcx=(speye(m*n)-kron(speye(n),circshift(eye(m),[1,0])))/hx;
Dcy=(speye(m*n)-kron(circshift(eye(n),[1,0]),speye(m)))/hy;
Dsq=mu*(Dcx*Dcx'+Dcy*Dcy'); OO=sparse(m*n,m*n);
A=[Dsq,OO,-Dcx';OO,Dsq,-Dcy';-Dcx,-Dcy,OO]; A(end,end)=A(end,end)+max(abs(A(:,end)));
bf=ones(m*n,1);b=rho*[gx*bf;gy*bf;0*bf]; ux=sparse(m,n);uy=sparse(m,n);p=sparse(m,n);
for i=1:n
    ux(i,m)=1;
end
isUx0=isSolid|circshift(isSolid,[-1 0]); isUy0=isSolid|circshift(isSolid,[0 -1]);
Rp=speye(m*n); Rp(isSolid(:,:))=[]; Rx=speye(m*n); Rx(isUx0(:,:))=[];
Ry=speye(m*n); Ry(isUy0(:,:))=[]; R=blkdiag(Rx,Ry,Rp);
ux=ux(:); uy=uy(:); p=p(:); uxb=-Dsq*ux; uyb=-OO*ux; OG=[uxb;uyb;p]; x=R*((R'*A*R')\ (R*(b+OG)));
ux=reshape(x(1:m*n),[m,n]); uy=reshape(x(m*n+1:2*m*n),[m,n]); p=reshape(x(2*m*n+1:3*m*n),[m,n]);
for i=1:n
    ux(i,m)=1;
end
toc
```